JSPM's Rajarshi Shahu College of Engineering, Tathawade, Pune
(An Autonomous Institute affiliated to Savitribai Phule Pune University)
Department of Computer Science and Business Systems

FoundIt

A Campus Lost and Found Web Application

Matching lost and found reports automatically, and verifying ownership before anything changes hands

---

Prasad Bhalerao (RBT24CB026)

Course Coordinator: Prof. Rutuja S. Bachhav

Skill Development Lab-II | Academic Year 2026-27

Index

1. Introduction
2. Problem Definition & Objectives
3. Literature Survey
4. Feasibility Study
5. System Design & Architecture
6. Technology & Tools Used
7. Implementation & Results
8. Conclusion & References

Introduction

What the project is, and why the problem is worth solving

A campus loses and finds things every day. Wallets, identity cards, earphones and hostel keys change hands constantly, and most lost items are found by somebody within hours.

The difficulty is not a shortage of honest finders. It is that the person who lost an object and the person holding it have no dependable way to find each other.

FoundIt is a web application that closes that gap, and then answers the harder question that follows: how do we know the person collecting the item is actually its owner?

Why it matters

- Shared spaces concentrate thousands of people and their belongings.
- Recovery today depends on chance encounters and word of mouth.
- Nothing records who received an item, or on what basis.

Problem Definition & Objectives

Existing approaches fail in two distinct ways, and a system must address both.

1. Discovery

Two reports describing the same object never meet, because nothing compares them.

2. Verification

Publishing enough detail to be recognised also teaches a false claimant exactly what to say.

Objectives

#	Objective
1	Centralise lost and found reporting in one searchable board
2	Identify likely matches automatically, and explain every suggestion
3	Verify ownership through a structured claim before any handover
4	Keep the detail that proves ownership out of public view
5	Record every report, decision and return for accountability

Literature Survey

How the problem is handled today, and what is missing

Existing approach	How it works	Limitation
Physical notice board	Handwritten slips in a corridor	Seen only on location; never updated when an item is recovered
Security desk register	Found items logged in a book	Searchable only by asking staff, during office hours
Class / hostel chat groups	Messages forwarded to a group	Reaches one cohort; scrolls out of view within a day
Generic lost-found portals	Keyword search over listings	Matching left to the user; no ownership verification

Gaps identified

- Discovery is manual - nothing proposes that two reports describe the same object
- Verification is absent - systems stop at putting two people in contact
- No auditable record of what was returned, to whom, or on what basis

Feasibility Study

Technical

- Built entirely on mature, documented technology: Jakarta EE 10, MySQL 8, GlassFish 7.
- No external services or paid APIs; the JDK supplies the cryptography.
- Runs on a single laboratory machine, deployed as one WAR file.

Financial / Business

- Every component is free and open source, so licensing cost is zero.
- Hosting is a single existing server; no recurring infrastructure spend.
- Value is recovered belongings and staff time saved at the security desk.

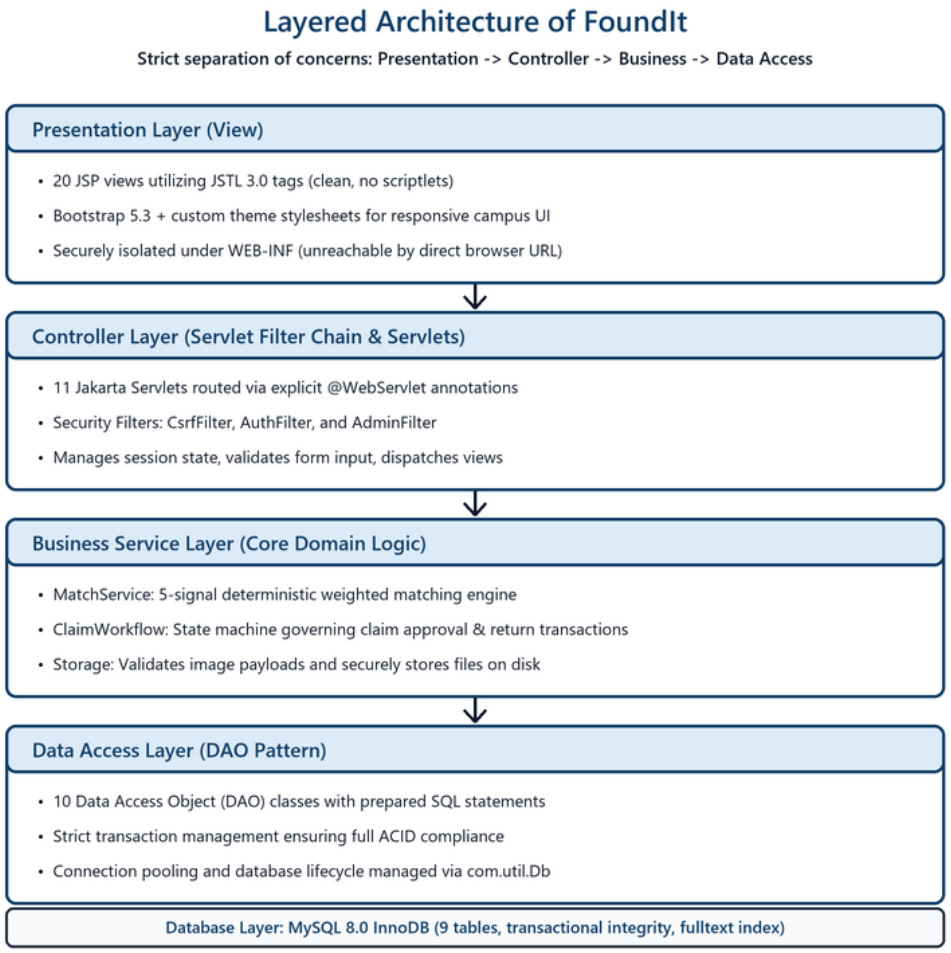
Operational

- Students already own the workflow; no staff member must mediate.
- The finder decides, so the institution carries no verification burden.
- Administrators moderate by exception rather than handling every case.

Conclusion: the project is feasible on all three counts and was built and deployed within the course timeline.

System Design & Architecture

Layered design: no layer reaches past the one below it

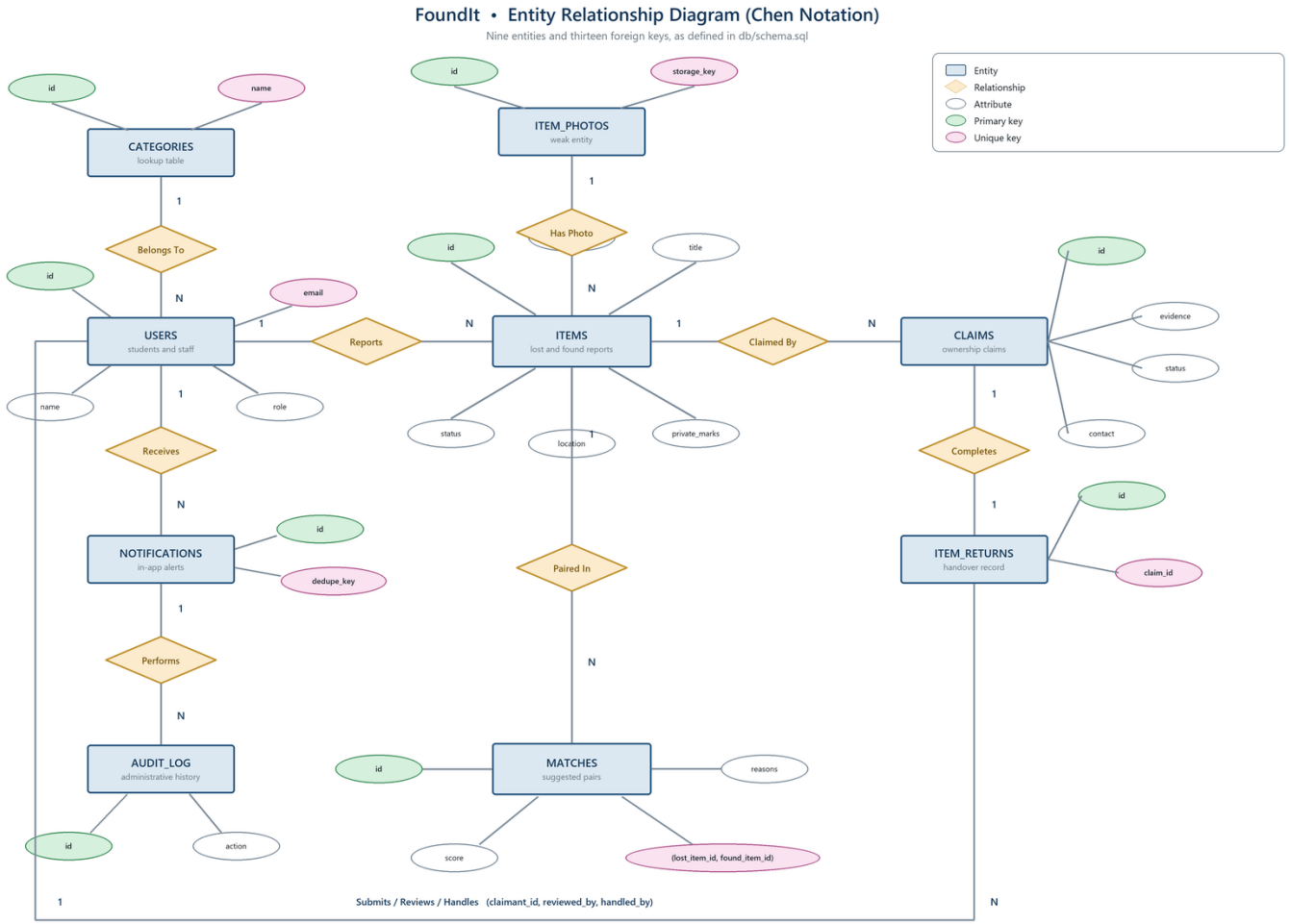


Design rules

- A view never touches a database connection.
- A data access class never writes HTML.
- Views live under WEB-INF, so no page can be reached by URL without passing its servlet.
- Business rules sit in services, not scattered across servlets.

Database Design

Nine entities in third normal form, thirteen foreign keys



Cardinality is read from the foreign keys: ITEM_RETURNS.claim_id carries a unique key, so CLAIMS to ITEM_RETURNS is one to one. MATCHES references ITEMS twice, once as lost_item_id and once as the found_item_id, and the pair is unique, which makes recomputation idempotent.

Constraints that carry logic

matches

A unique key on the (lost, found) pair makes recomputation idempotent: scores refresh, duplicates cannot accumulate.

notifications

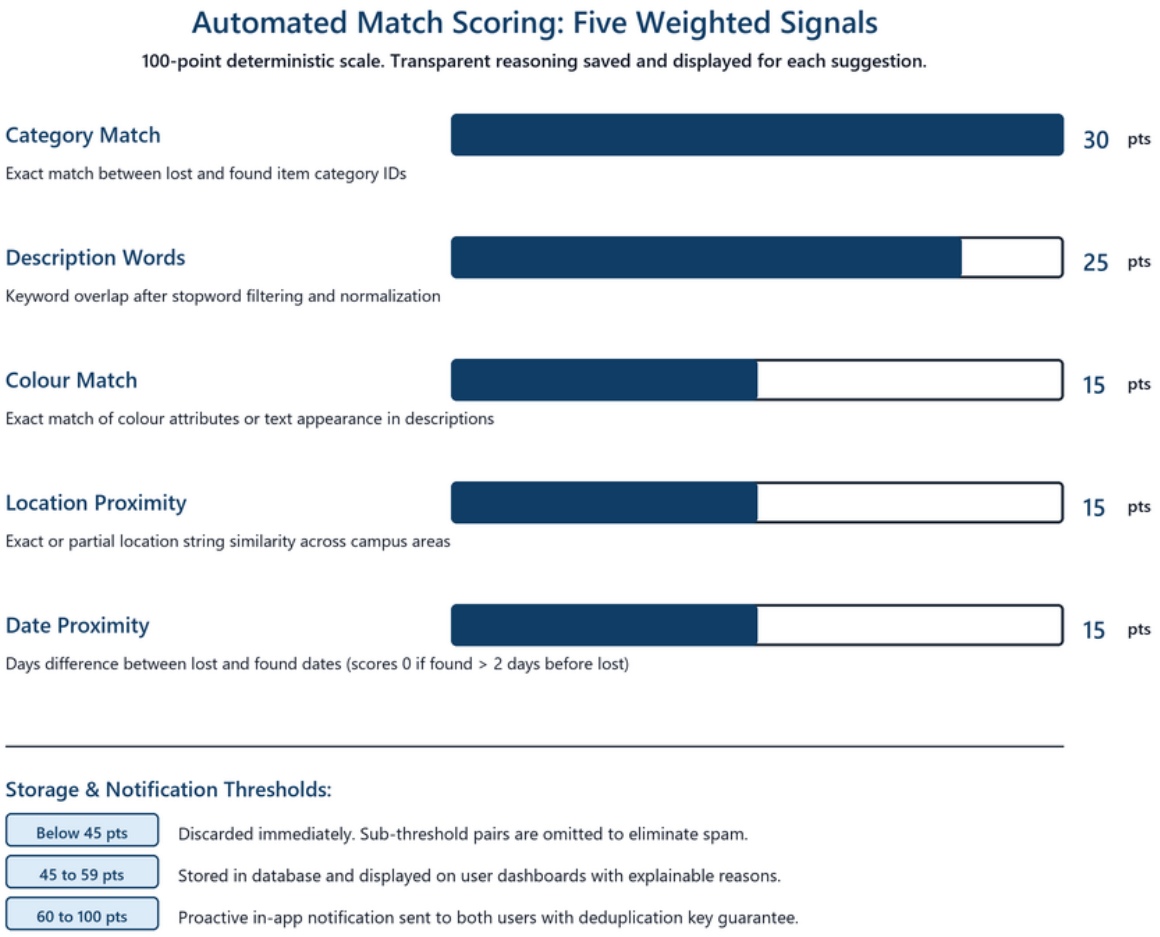
A unique dedupe key makes notify-once a guarantee of the database, not a hope of the application.

deletion

Claims and returns refuse deletion, which forces the system to close and deactivate rather than destroy history.

The Matching Engine

A deterministic weighted score, and the reason behind it



Worked example

Lost: hostel key, red tag, sports ground

Found: two keys, red tag, ground gate, next day

Same category +30

Shared words +22

Same colour +15

Nearby place +10

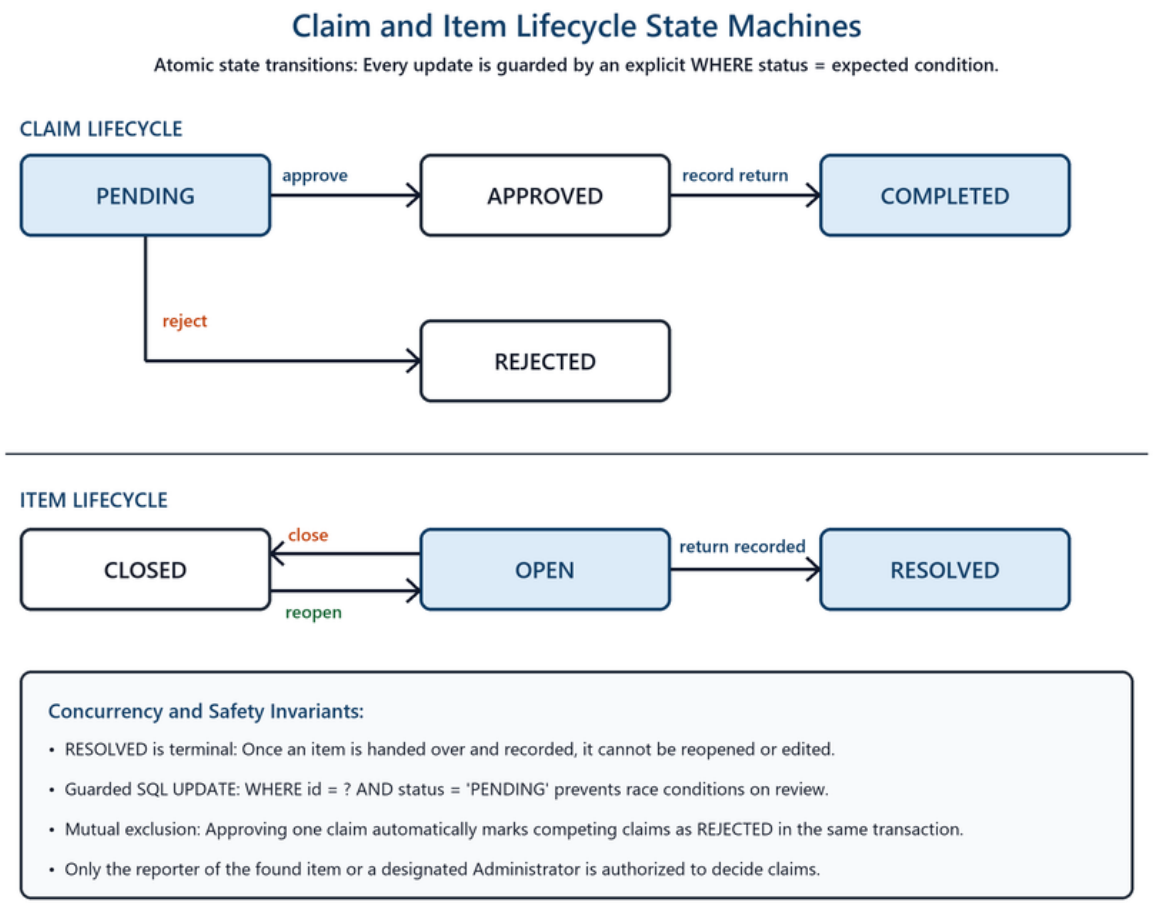
One day apart +15

92 / 100 Strong

The five reasons are stored with the match and shown to the user.

Claim & Return Workflow

The algorithm proposes; a person decides



Guarantees

- Every transition is guarded by the status it comes from, so an illegal jump cannot happen.
- Two reviewers clicking at once cannot both succeed.
- Approving one claim rejects the others, so two people are never both told they are the owner.
- A return runs as one transaction across five tables.

Technology & Tools Used

Layer	Technology
IDE	Apache NetBeans (Ant)
Language	Java 17
Web	Jakarta EE 10, Servlet 6.0, JSP, JSTL 3.0
Server	GlassFish 7.1.1
Database	MySQL 8.0, Connector/J 9.7
Front end	HTML5, CSS3, Bootstrap 5
Security	PBKDF2-HMAC-SHA256 (JDK)

Security measures

- Passwords hashed with PBKDF2, 120,000 iterations, per-user salt
- CSRF token required on every state-changing request
- Every SQL statement prepared; no query built from user input
- All user content escaped on output
- Role fixed server-side, so no form can create an administrator
- Uploads must decode as an image; filenames never trusted
- Photographs served by id, never by path

Implementation & Results

41	20	9	5	100%
Java classes	JSP views	database tables	scoring signals	match precision

Functional Verification

Workflow Module	Observed Outcome
Authentication & RBAC	Role checks & session security verified
Lost & Found Reporting	Multipart image upload & validation
5-Signal Matching	Automatic scoring on item publish
Claim & Evidence Review	Private marks withheld from public
Handover & Return	Atomic return logging & audit trail

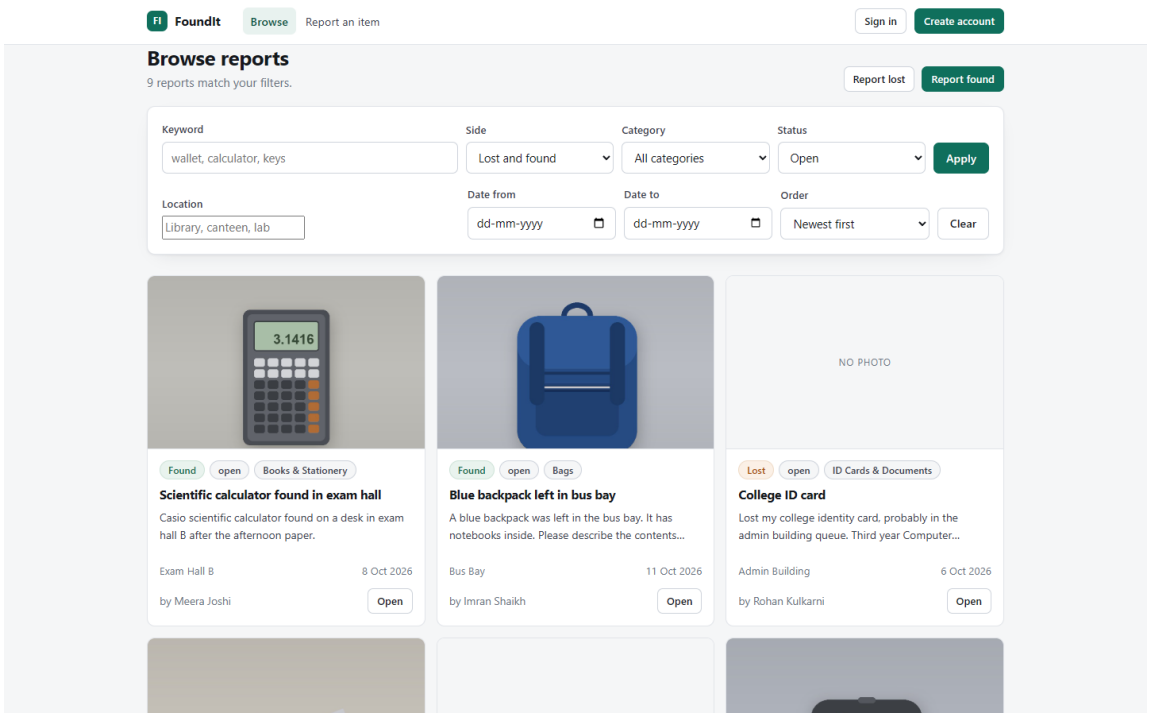
Matching accuracy on seeded data

Candidate Pair	Score
Hostel keys <-> keys at ground gate	92%
Earphones case <-> case on lab table	86%
Brown wallet <-> wallet at canteen	76%
Unmatched reports (ID, calculator)	No false match

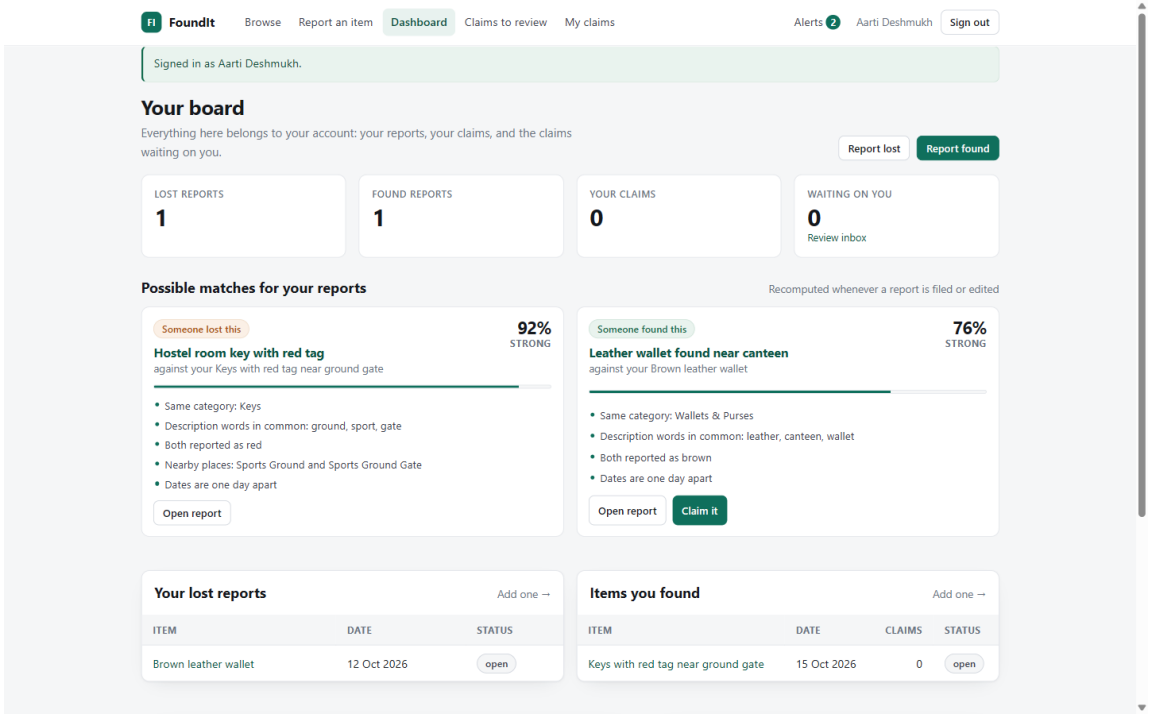
Three genuine pairs identified; zero false positives.

The Application

Running on GlassFish 7.1.1 with the seeded dataset



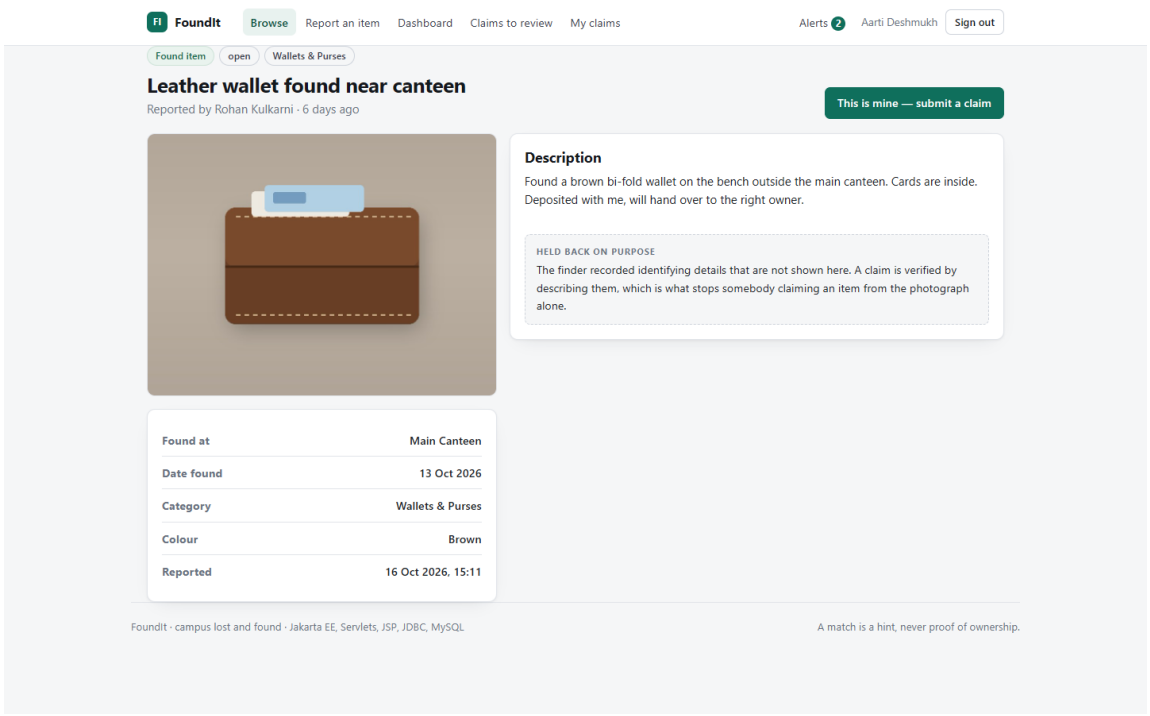
Browse: filters and found-item reports



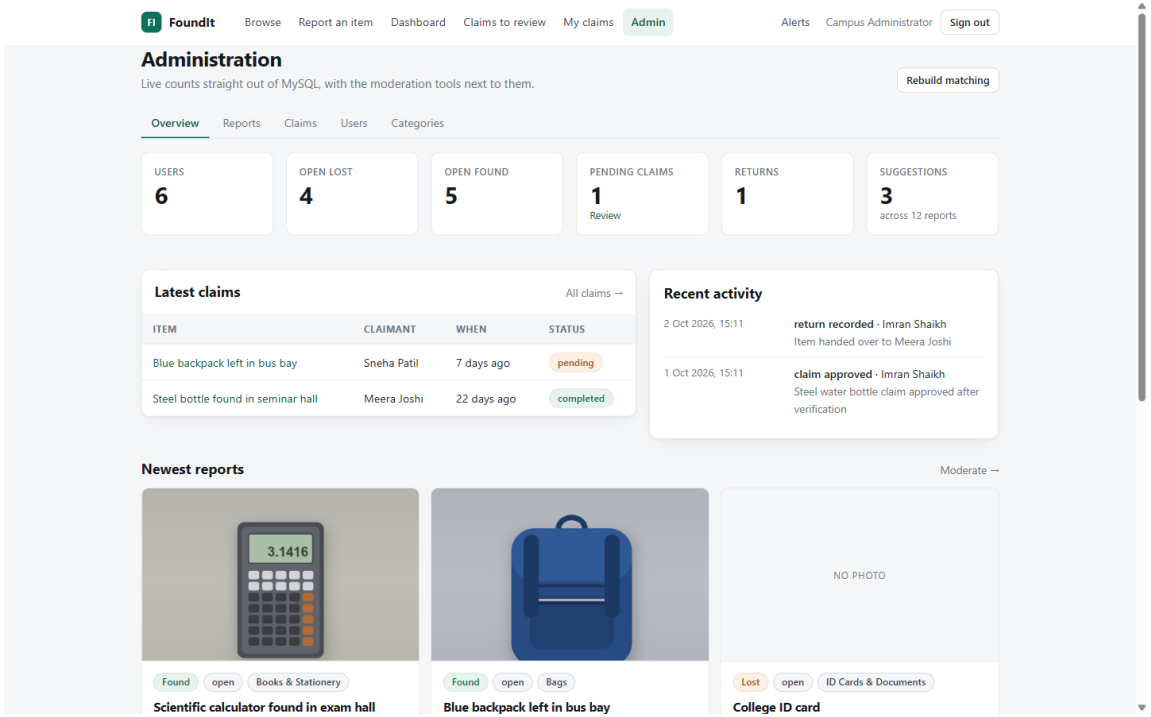
Dashboard: scored match suggestions

Verification in the Interface

The identifying detail is withheld, and the finder decides



Report detail: the held-back identifying detail



Administration: counts, claims and audit trail

Conclusion

FoundIt shows that a campus lost-and-found service becomes markedly more useful when two capabilities are added to simple listing:

automatic matching, so the two halves of a case find each other without anybody searching, and structured verification, so an item is released only to someone who can describe what the public page does not show.

Correctness guarantees were placed in the layer best able to enforce them: uniqueness and deduplication in the database, status transitions inside the SQL that performs them, and authorisation on the server for the specific record being touched.

Future scope

- Email and push notification delivery
- Image similarity as a sixth matching signal
- Full-text relevance using the existing index
- Photograph attachments on claims
- QR-coded handover receipts
- REST API and a mobile client

References

[1] M. Al-Mousa et al., Web-Based Lost and Found Information System for University Environments, IEEE ICCIS, 2020.

[2] S. R. Bharambe et al., Automated Lost and Found Management System Using Attribute Matching, IJARCCE, 2021.

[3] Jakarta EE 10 Platform Specification, Eclipse Foundation Standards, 2022.

[4] Jakarta Servlet 6.0 Specification, Eclipse Foundation Standards, 2022.

[5] MySQL 8.0 Reference Manual: InnoDB Storage Engine, Oracle Corporation, 2024.

[6] P. Christen, Survey of Indexing Techniques for Scalable Record Linkage, IEEE TKDE, 2012.

[7] OWASP Top 10 Web Application Security Risks, OWASP Foundation, 2021.

[8] Recommendation for Password-Based Key Derivation: PBKDF2, NIST SP 800-132, 2010.

Thank you

Questions are welcome

Prasad Bhalerao (RBT24CB026) | github.com/prasadbhalerao1/FoundIt_SDL